INTRODUCTION AU CLOUD COMPUTING

TP 2

Premiers conteneurs avec Docker

Compte rendu

Classe	L3 SI — RIST & BDGL
Séance	Séance 3 — Virtualisation : VM et conteneurs
Étudiants	Coumare Sitan Founin
	Konan Kouassi Henri Joel
	Yeo Gnenakan Ibrahima
Date	30 octobre 2026

Objectif : installer Docker dans la machine virtuelle Ubuntu Server, exécuter et inspecter des conteneurs, comparer leur fonctionnement à celui d'une machine virtuelle, conteneuriser une API Flask puis publier son image dans un registre.

Introduction

Dans le TP1, nous avons créé une machine virtuelle Ubuntu Server avec VirtualBox. Ce TP2 poursuit le travail en utilisant Docker à l'intérieur de cette VM. Contrairement à une VM qui embarque un système complet, un conteneur partage le noyau Linux de la machine hôte et ne contient que l'application et ses dépendances.

L'architecture comporte deux services : un serveur web Nginx accessible sur le port 8080 de la VM et une API Flask accessible sur le port 5000. Depuis Windows, les accès passent d'abord par les redirections de ports VirtualBox, puis par les ports publiés par Docker.

Chemin d'accès aux services

- Navigateur Windows → 127.0.0.1:8080 → redirection VirtualBox vers Ubuntu Server:8080 → publication Docker 8080:80 → Nginx.
- Navigateur Windows → 127.0.0.1:5000 → redirection VirtualBox vers Ubuntu Server:5000 → publication Docker 5000:5000 → API Flask.

Partie 0 — Préparer la VM et installer Docker

0.1 Démarrage de la VM et redirections de ports

Depuis PowerShell sous Windows, nous démarrons la VM en mode sans interface, puis nous créons les deux redirections de ports :

```
$env:Path += ";C:\Program Files\Oracle\VirtualBox"
VBoxManage startvm ubuntu-srv --type headless
VBoxManage controlvm ubuntu-srv natpf1 "web,tcp,127.0.0.1,8080,,8080"
VBoxManage controlvm ubuntu-srv natpf1 "api,tcp,127.0.0.1,5000,,5000"
ssh -p 2222 etudiant@127.0.0.1
```

La connexion SSH utilise le port 2222 configuré au TP1. Les redirections permettent à Windows de joindre les services qui écoutent sur les ports 8080 et 5000 de la VM.

0.2 Installation de Docker

```
sudo apt update
sudo apt install -y docker.io docker-buildx docker-compose-v2
sudo usermod -aG docker $USER
exit
```

Après la déconnexion, nous nous reconnectons en SSH pour que l'appartenance au groupe docker soit prise en compte, puis nous vérifions l'installation :

```
docker version
docker info | grep -E "Server Version|Storage Driver|Cgroup|Kernel Version"
```

Q1. Quelle version de Docker est installée ? Quel noyau Docker utilise-t-il ?

La commande docker version indique Docker Engine 29.1.3 (API 1.52, Go 1.24.13), avec containerd 2.2.2, runc 1.4.0 et docker-init 0.19.0. Docker utilise le noyau de la VM : Linux 7.0.0-38-generic, affiché par la bannière de connexion et confirmé par uname -r (voir Partie 1). La VM tourne sous Ubuntu Server 26.04.1 LTS, avec le pilote de cgroups systemd (cgroup version 2).

Q2. Pourquoi le groupe docker équivaut-il presque à être administrateur ? Quelle précaution prendre sur un serveur partagé ?

Un membre du groupe docker peut communiquer avec le démon Docker, qui s'exécute avec les droits root. Il peut donc lancer un conteneur avec des privilèges élevés et monter des répertoires sensibles de la machine hôte, ce qui lui donne un contrôle proche de celui d'un administrateur. Sur un serveur partagé, il faut limiter strictement l'accès à ce groupe, ne l'accorder qu'à des utilisateurs de confiance et éviter d'y ajouter des comptes sans nécessité.

Preuves des manipulations

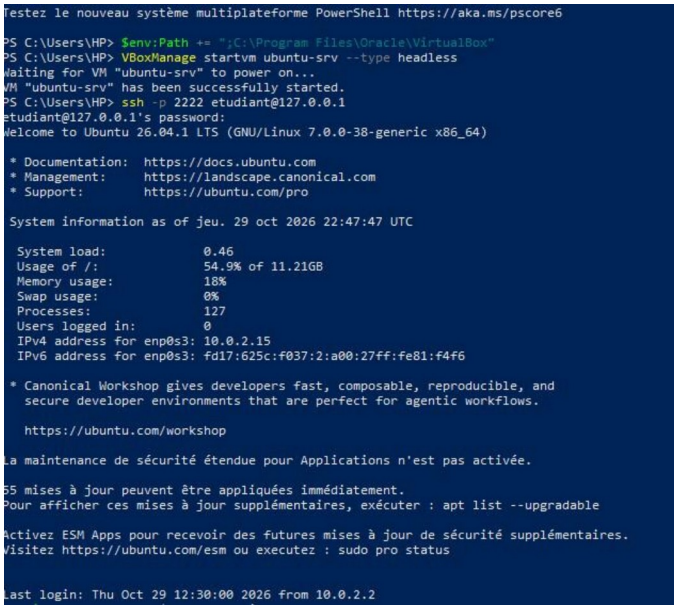


Figure 1 — Démarrage de la VM Ubuntu Server et connexion SSH

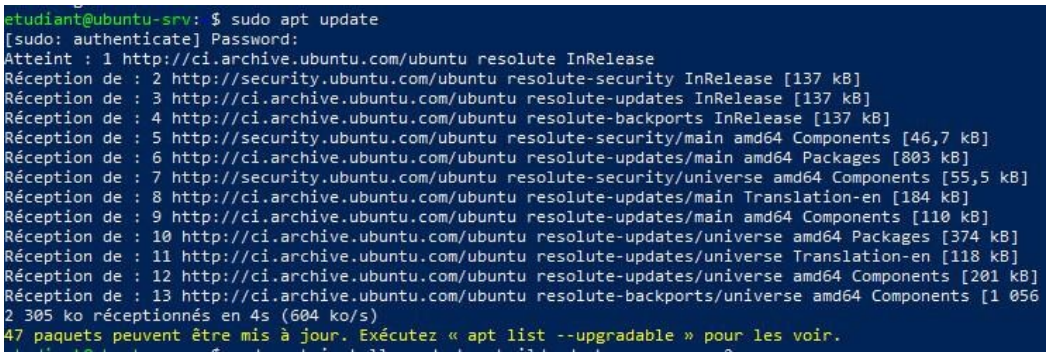


Figure 2 — Mise à jour de l'index des paquets (sudo apt update)

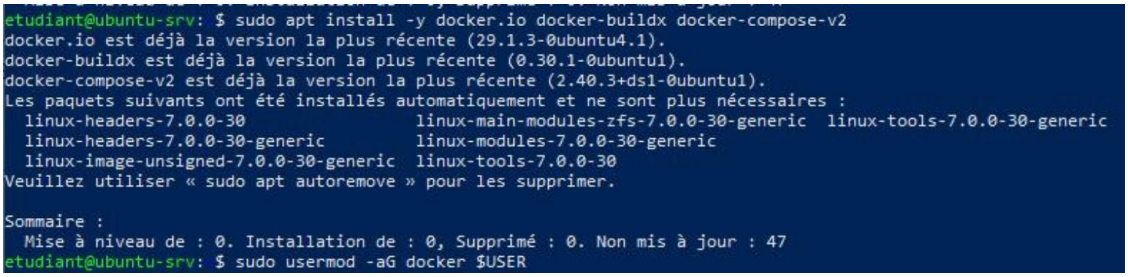


Figure 3 — Installation de Docker, Buildx et Compose

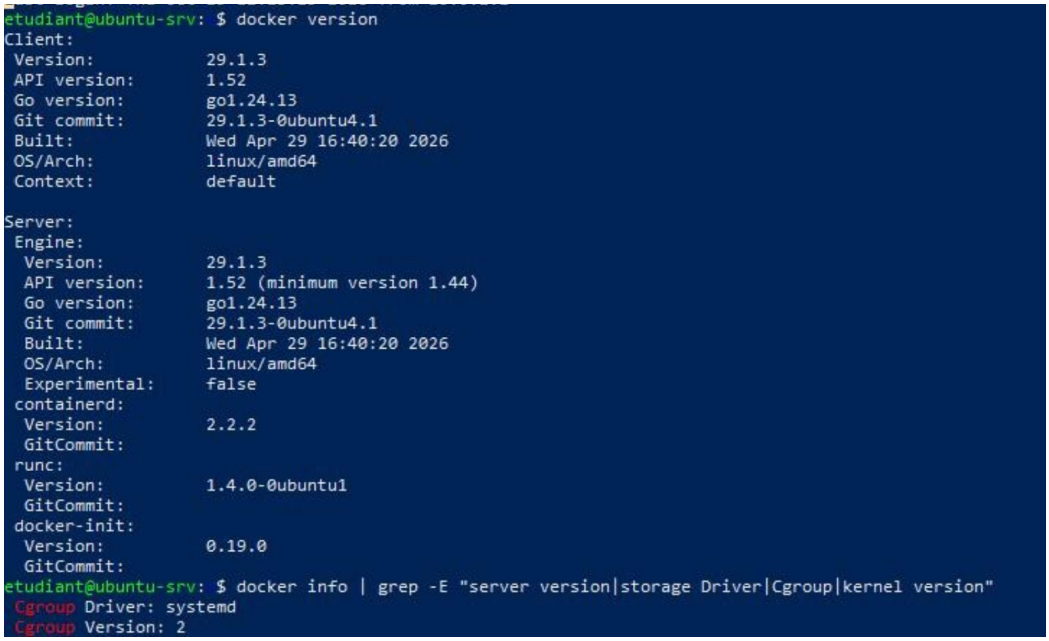


Figure 4 — Résultat de docker version et informations du moteur

Partie 1 — Lancer et inspecter des conteneurs

1.1 Premier conteneur

```
docker run hello-world
docker images
docker ps -a
```

Docker vérifie d'abord si l'image hello-world est disponible localement. Sinon, il la télécharge depuis le registre, crée un conteneur à partir de cette image et exécute le programme qui affiche le message de bienvenue. Le conteneur se termine ensuite : il apparaît dans docker ps -a, mais plus dans docker ps.

Q1. Quelles sont les quatre étapes réalisées par Docker selon le message hello-world ?

1) Le client Docker contacte le démon Docker. 2) Le démon télécharge l'image « hello-world » depuis Docker Hub (version amd64). 3) Le démon crée un conteneur à partir de cette image, qui exécute le programme produisant le message. 4) Le démon transmet la sortie au client Docker, qui l'affiche dans le terminal.

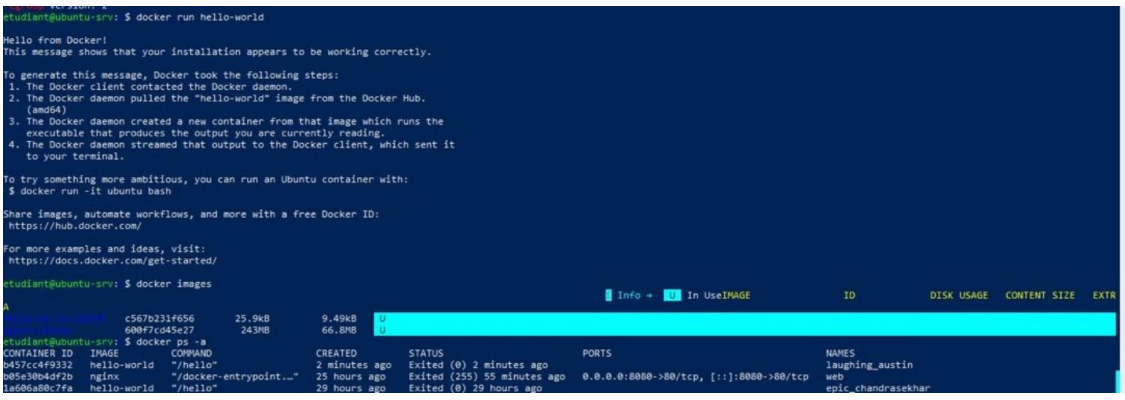


Figure 5 — Exécution de hello-world, liste des images et des conteneurs

1.2 Lancer un serveur web Nginx

```
docker run -d --name web -p 8080:80 nginx
```

```
docker ps
curl -s http://localhost:8080 | head -n 5
```

L'option -d lance le conteneur en arrière-plan, --name web lui donne un nom, et -p 8080:80 relie le port 8080 de la VM au port 80 du conteneur, sur lequel Nginx écoute. Depuis Windows, la page « Welcome to nginx! » s'affiche à l'adresse http://127.0.0.1:8080.

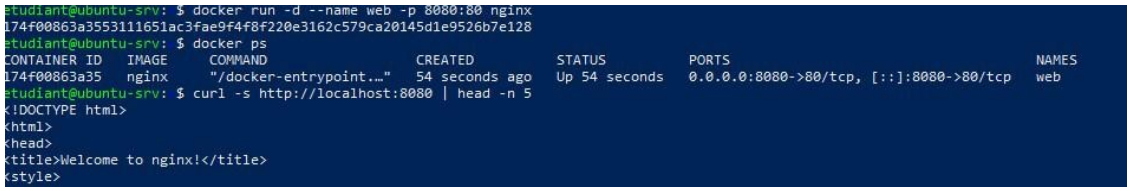


Figure 6 — Lancement du conteneur Nginx, docker ps et test avec curl



Figure 7 — Page « Welcome to nginx! » affichée depuis Windows sur 127.0.0.1:8080

1.3 Observer l'intérieur du conteneur

```
docker exec web cat /etc/os-release | head -n 2
cat /etc/os-release | head -n 2
docker exec web uname -r
uname -r
docker logs --tail 5 web
docker top web
ps aux | grep "nginx: master" | grep -v grep
docker stats --no-stream web
```

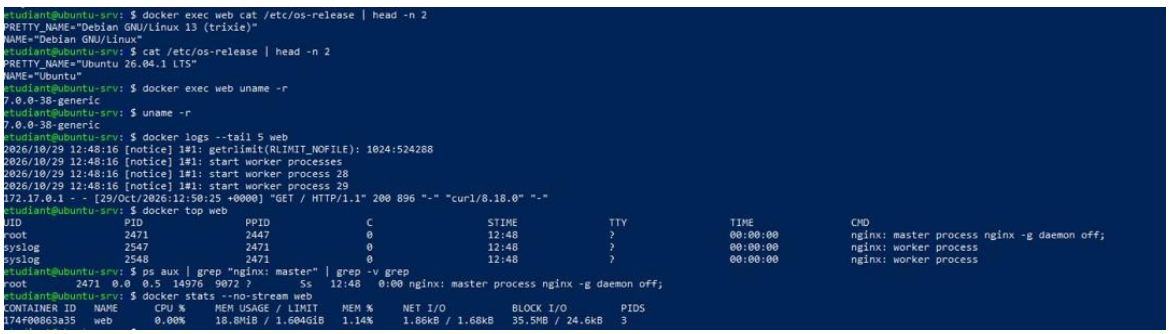


Figure 8 — os-release, uname -r, logs, docker top, ps aux et docker stats

Q2. Le conteneur web et la VM ont-ils la même distribution et le même noyau ?

Non pour la distribution, oui pour le noyau. Le conteneur Nginx tourne sur Debian GNU/Linux 13 (trixie), alors que la VM est sous Ubuntu 26.04.1 LTS. En revanche, uname -r renvoie 7.0.0-38-generic

des deux côtés : le conteneur partage le noyau de la VM. C'est la différence essentielle entre conteneurisation et virtualisation matérielle.

Q3. Les processus Nginx apparaissent-ils dans ps aux sur la VM ? Que montre cette observation ?

Oui. Le processus « nginx: master process » (PID 2471) apparaît à la fois dans docker top et dans ps aux sur la VM, avec ses processus workers. Un conteneur n'est donc pas une machine virtuelle indépendante avec son propre noyau : c'est un ensemble de processus du noyau Linux de la VM, isolés grâce aux mécanismes du noyau (espaces de noms et cgroups). docker stats indique 18,8 MiB de mémoire utilisée, 0,00 % de CPU et 3 processus.

Q4. Quel chemin suit une requête du navigateur jusqu'à Nginx ?

Le navigateur Windows contacte 127.0.0.1:8080. VirtualBox redirige cette requête vers le port 8080 de la VM. Docker redirige ensuite ce port vers le port 80 du conteneur web, où Nginx reçoit et traite la requête.

1.4 Caractère éphémère du conteneur et volume

```
docker exec web sh -c 'echo "<h1>Trinôme ...</h1>" >
/usr/share/nginx/html/index.html'
curl -s http://localhost:8080
docker rm -f web
docker run -d --name web -p 8080:80 nginx
curl -s http://localhost:8080 | grep -o "<title>.*</title>"
```

La modification est faite dans le système de fichiers propre au conteneur. Quand celui-ci est supprimé avec docker rm -f, elle disparaît : le nouveau conteneur est créé à partir de l'image Nginx d'origine et retrouve la page « Welcome to nginx! ». Dans notre capture, la première tentative a échoué car le chemin /usr/share/html n'existe pas ; nous l'avons corrigée avec /usr/share/nginx/html.

```
mkdir -p ~/site
echo "<h1>Trinôme ... : servi depuis un volume</h1>" > ~/site/index.html
docker rm -f web
docker run -d --name web -p 8080:80 -v ~/site:/usr/share/nginx/html:ro nginx
curl -s http://localhost:8080
```

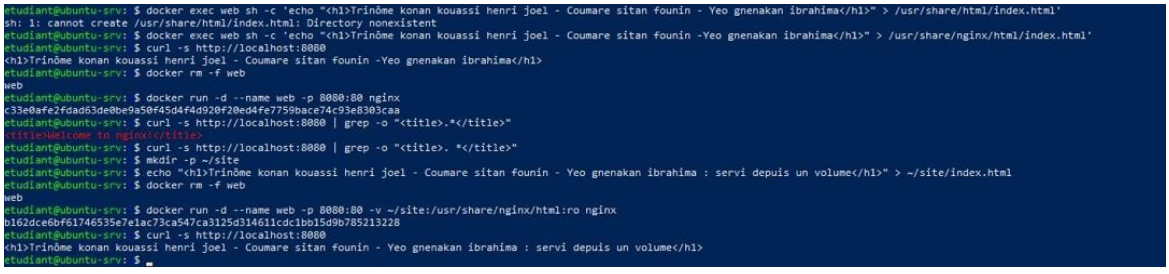


Figure 9 — Page modifiée dans le conteneur (perdue après docker rm -f), puis page servie depuis un volume

Q5. Pourquoi la page modifiée a-t-elle disparu après docker rm ? Que change l'option -v ?

La page modifiée était enregistrée dans la couche inscriptible du conteneur, qui est supprimée avec lui. L'option -v ~/site:/usr/share/nginx/html:ro monte le dossier ~/site de la VM dans le répertoire web de Nginx : les fichiers sont conservés sur la VM indépendamment du cycle de vie du conteneur. Le suffixe :ro rend le montage accessible en lecture seule depuis le conteneur. Après le montage, curl renvoie bien notre page personnalisée.

Partie 2 — Comparer conteneur et machine virtuelle

2.1 Mesures effectuées

```
docker pull alpine
docker images
time docker run --rm alpine echo "Bonjour depuis un conteneur"
time docker run --rm alpine echo "Bonjour depuis un conteneur"
systemd-analyze
free -h
```

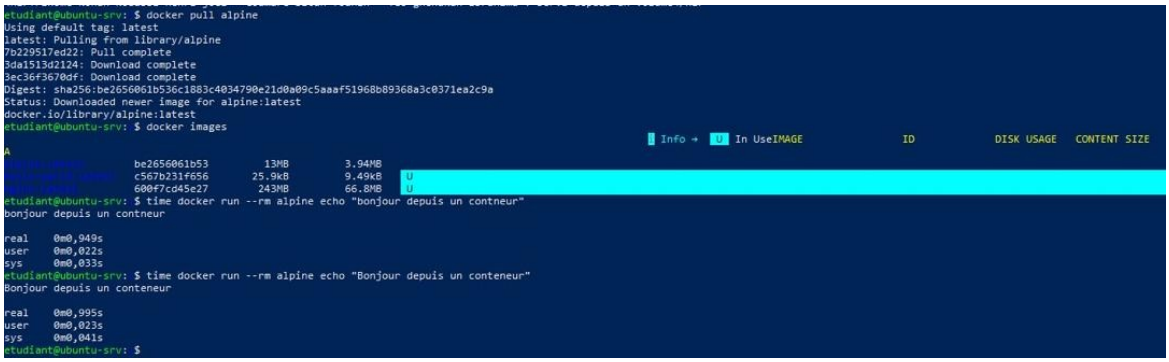


Figure 10 — docker pull alpine, docker images et deux exécutions de time docker run



Figure 11 — Temps de démarrage de la VM (systemd-analyze) et mémoire (free -h)

2.2 Tableau comparatif

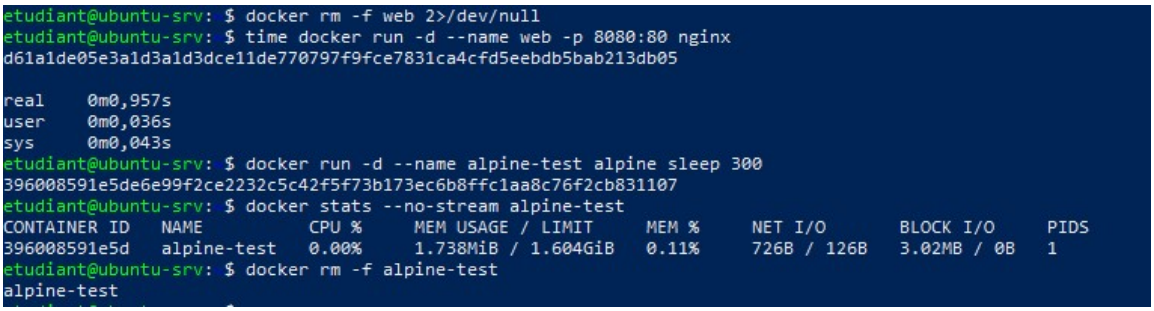


Figure 11b — Temps de démarrage de nginx (time docker run -d) et mémoire d'alpine (docker stats)

Mesure	VM ubuntu-srv	Conteneur nginx	Conteneur alpine
Taille sur disque	≈ 6,2 Go utilisés (54,9 % de 11,21 Go)	243 MB (contenu : 66,8 MB)	13 MB (contenu : 3,94 MB)
Temps de démarrage	23,012 s (noyau 1,700 s + initrd 6,473 s + espace utilisateur 14,839 s)	0,957 s (time docker run -d)	0,995 s (2e commande time)
Mémoire utilisée	445 Mi sur 1,6 Gi	18,8 MiB	1,738 MiB
Système	Ubuntu Server 26.04.1 LTS + noyau 7.0.0-38	Debian GNU/Linux 13 (trixie), noyau de la VM	Alpine, noyau de la VM

Q1. Dans quel rapport le conteneur est-il plus léger et plus rapide que la VM ? Les mesures confirment-elles le cours ?

Taille : l'espace occupé par la VM (≈ 6,2 Go) est environ 25 fois supérieur à l'image nginx (243 MB) et environ 470 fois supérieur à l'image alpine (13 MB). Temps : la VM démarre en 23,012 s contre 0,995 s pour le conteneur alpine, soit environ 23 fois plus vite. Mémoire : la VM utilise 445 Mi contre 18,8 MiB pour le conteneur nginx, soit environ 24 fois moins.

Les mesures confirment le cours : un conteneur est nettement plus léger et plus rapide à démarrer qu'une VM, puisqu'il ne démarre ni noyau ni système complet.

Q2. Pourquoi le premier time docker run est-il plus long que le second ?

En général, le premier lancement peut inclure le téléchargement de l'image depuis le registre, ce qui ajoute du temps réseau et de stockage ; au second lancement, l'image est déjà présente localement. Dans notre cas, l'image alpine avait déjà été téléchargée par docker pull alpine : les deux mesures sont donc très proches (0,949 s et 0,995 s) et la différence reste de l'ordre de la variation normale d'une mesure à l'autre.

Q3. Citez un cas où vous choisiriez une VM plutôt qu'un conteneur.

Nous choisirions une machine virtuelle lorsqu'il faut exécuter un système d'exploitation différent ou disposer d'un noyau indépendant, par exemple pour tester une autre distribution Linux ou un système Windows, ou lorsqu'une isolation plus forte au niveau du système est nécessaire.

Partie 3 — Conteneuriser l'API fil rouge

3.1 Création de l'application Flask

```
mkdir -p ~/api && cd ~/api
cat > app.py <<'EOF'
import os
import socket
from flask import Flask, jsonify
app = Flask(__name__)

@app.get("/")
def accueil():
    return jsonify(
        message="API fil rouge",
        trinome=os.environ.get("TRINOME", "inconnu"),
        conteneur=socket.gethostname(),
    )

@app.get("/sante")
def sante():
    return jsonify(statut="ok")

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
EOF
echo "flask>=3,<4" > requirements.txt
```

La route / renvoie un message, le nom du trinôme transmis par variable d'environnement et le nom d'hôte du conteneur. La route /sante permet de vérifier simplement que l'API répond.



Figure 12 — Création de app.py et de requirements.txt dans ~/api

3.2 Dockerfile et .dockerignore

```
FROM python:3.12-slim
WORKDIR /app
```

```
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

FROM choisit l'image Python de base ; WORKDIR définit le répertoire de travail ; COPY copie les fichiers ; RUN installe Flask ; EXPOSE documente le port utilisé par l'application ; CMD définit la commande de démarrage. Le fichier .dockerignore exclut les fichiers Python temporaires et le dossier Git du contexte de construction.

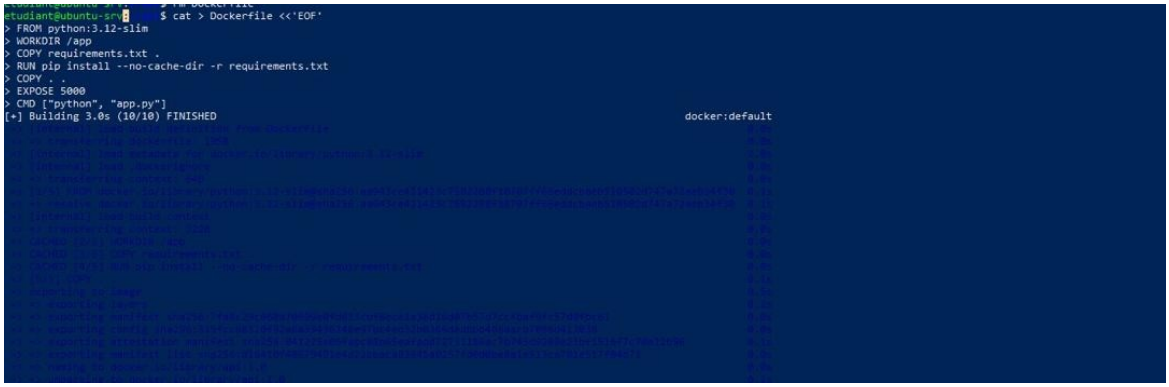


Figure 13 — Création du Dockerfile et construction de l'image api:1.0

3.3 Construction et inspection de l'image

```
docker build -t api:1.0 .
docker images api
docker history api:1.0
```

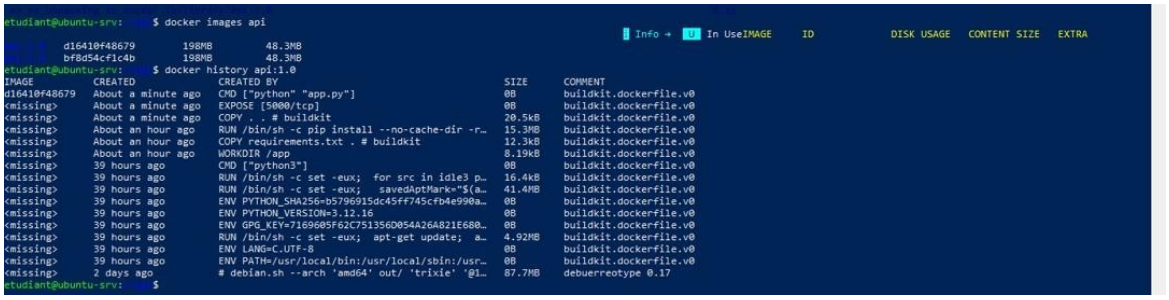


Figure 14 — docker images api et docker history api:1.0

3.4 à 3.6 Lancement, trois copies et nouvelle version

```
docker run -d --name api -p 5000:5000 -e TRINOME="..." api:1.0
curl -s http://localhost:5000
curl -s http://localhost:5000/sante
docker logs api

for p in 5001 5002 5003; do
  docker run -d --name api-$p -p $p:5000 -e TRINOME="..." api:1.0
done
for p in 5001 5002 5003; do curl -s http://localhost:$p; echo; done
docker rm -f api-5001 api-5002 api-5003

# ajout de la route /version dans app.py, puis :
docker build -t api:1.1 .
docker rm -f api
docker run -d --name api -p 5000:5000 api:1.1
curl -s http://localhost:5000/version
```

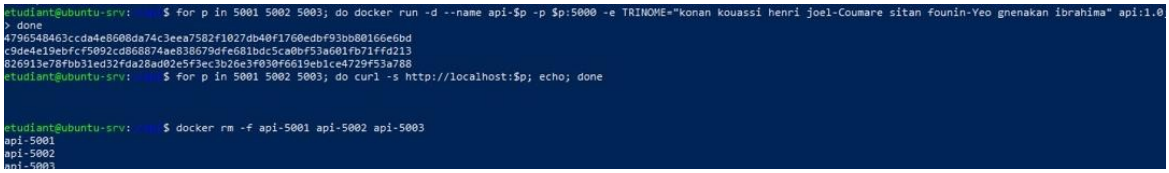


Figure 15 — Lancement de trois conteneurs api-5001, api-5002 et api-5003, puis suppression

Q1. Quelle est la taille de api:1.0 ? Combien de couches affiche docker history et lesquelles viennent du Dockerfile ?

L'image api:1.0 occupe 198 MB sur disque (48,3 MB de contenu). docker history affiche 16 lignes. Les six premières viennent de notre Dockerfile : WORKDIR /app (8,19 kB), COPY requirements.txt (12,3 kB), RUN pip install (15,3 MB), COPY . . (20,5 kB), puis EXPOSE 5000 et CMD, de taille nulle car ce sont des métadonnées. Les dix lignes suivantes appartiennent à l'image de base python:3.12-slim (Debian trixie), dont la couche système de 87,7 MB.

Q2. Pourquoi copier requirements.txt et installer les dépendances avant de copier le code ?

Cette organisation exploite le cache Docker. Tant que requirements.txt ne change pas, la couche qui installe Flask reste valide et est réutilisée. Modifier app.py ne force donc pas à réinstaller les dépendances, ce qui accélère les reconstructions : les étapes WORKDIR, COPY requirements.txt et RUN pip install sont marquées CACHED lors de la construction.

Q3. Que renvoie le champ conteneur pour les trois copies ? Quelle caractéristique du cloud cela illustre-t-il ?

Chaque copie renvoie un nom d'hôte différent, car le nom d'hôte d'un conteneur correspond à son identifiant, propre à chaque conteneur (ici 4796548463cc, c9de4e19ebfc et 826913e78fbb pour api-5001, api-5002 et api-5003). Cela illustre l'élasticité : on peut lancer plusieurs instances d'une même application à partir d'une image identique, chacune dans un environnement isolé, et en ajouter ou en retirer selon les besoins.

Q4. Pourquoi utiliser une variable d'environnement pour le nom du trinôme plutôt que l'écrire dans le code ? Quel lien avec les secrets ?

Une variable d'environnement permet de configurer l'application au lancement sans modifier ni reconstruire l'image : la même image sert dans plusieurs environnements avec des paramètres différents. Le principe est le même pour les secrets (mots de passe, clés d'API) : ils ne doivent jamais être écrits en dur dans le code ni intégrés dans une image, car les couches d'une image peuvent être inspectées par quiconque y a accès. En production, on utilise un mécanisme de gestion de secrets adapté.

Partie 4 — Publier l'image et nettoyer

4.1 Publication sur Docker Hub

```
docker login
docker tag api:1.1 kouassihenrijoelk08/api-fil-rouge:1.1
docker push kouassihenrijoelk08/api-fil-rouge:1.1
```

L'authentification se fait par activation de l'appareil : docker login affiche un code que l'on saisit sur la page web de Docker, puis la connexion est confirmée.

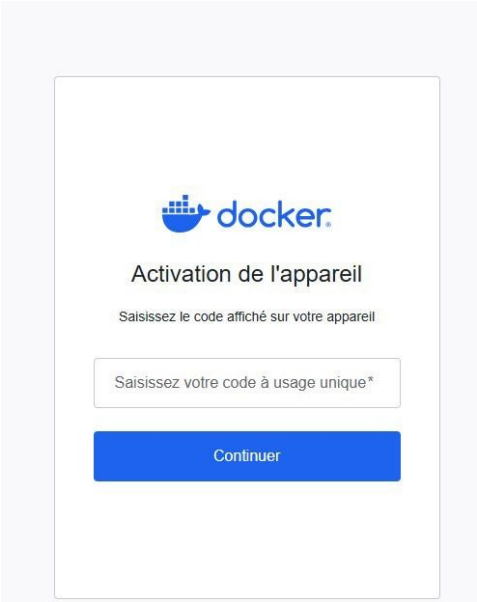


Figure 16 — Activation de l'appareil (docker login)

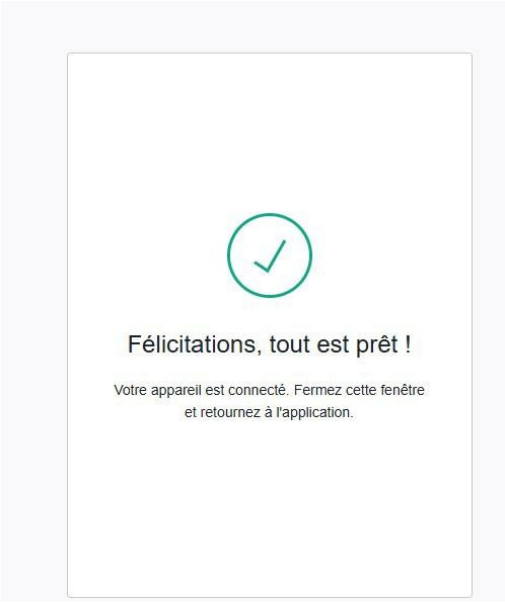


Figure 17 — Appareil connecté

```
etudiant@ubuntu-srv: $ docker login
Authenticating with existing credentials... [Username: kouassihenrijoelk08]

i Info → To login with a different account, run 'docker logout' followed by 'docker login'


Login Succeeded
etudiant@ubuntu-srv: $ docker tag api:1.1 kouassihenrijoelk08/api-fil-rouge:1.1
etudiant@ubuntu-srv: $ docker push kouassihenrijoelk08/api-fil-rouge:1.1
The push refers to repository [docker.io/kouassihenrijoelk08/api-fil-rouge]
acca7542dfc7: Pushed
6fc242d221a6: Mounted from library/nginx
b3e524f2e530: Pushed
3eb771151506: Pushed
c159e48e21d9: Pushed
8f774c1e0540: Pushed
e75b98e0d6c5: Pushed
3f2ec462bfdd: Pushed
f574edac59d9: Pushed
1.1: digest: sha256:9c7c10408ad715563326ffeb2dca83288f6346acb2dac1f8591afc893bbedaae size: 856
etudiant@ubuntu-srv: $
```

Figure 18 — Connexion, étiquetage (docker tag) et publication (docker push) de l'image api:1.1

4.2 Essai avec GitHub Container Registry

Nous avons aussi tenté de nous connecter à GHCR. La connexion a été refusée (« denied »), car GHCR demande un jeton GitHub disposant du droit write:packages. Nous avons donc poursuivi avec Docker Hub.

4.3 Vérification de l'image publiée

```
docker rm -f api
docker rmi kouassihenrijoelk08/api-fil-rouge:1.1 api:1.1
docker run -d --name api -p 5000:5000 kouassihenrijoelk08/api-fil-rouge:1.1
curl -s http://localhost:5000/version
```

Après suppression des images locales, Docker n'a pas trouvé l'image en local et l'a téléchargée depuis Docker Hub (« Unable to find image … locally », puis « Pull complete »), puis le conteneur a démarré. Cela confirme que l'image publiée est bien récupérable depuis le registre.

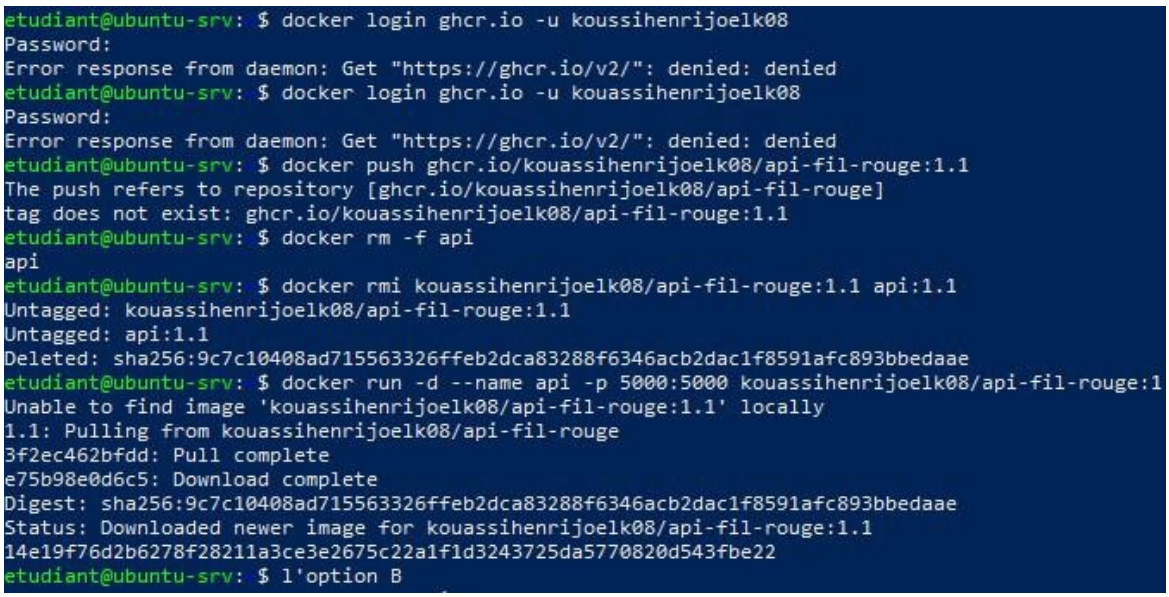


Figure 19 — Essai GHCR refusé, suppression des images locales et récupération depuis Docker Hub

4.4 Nettoyage

```
docker rm -f $(docker ps -aq)
docker system df
docker image prune -a -f
docker system df
```

La première commande supprime tous les conteneurs de la VM. Les deux docker system df affichent l'espace utilisé par Docker avant et après le nettoyage. docker image prune -a -f supprime les images inutilisées ; sur une machine partagée, il faut vérifier les ressources avant de l'exécuter.



Figure 20 — Nettoyage : docker rm -f, docker system df, docker image prune -a -f et docker system df

Q1. Quelle est l'adresse complète de l'image publiée ? Est-elle publique ou privée ?

L'image publiée est kouassihenrijoelk08/api-fil-rouge:1.1 sur Docker Hub (adresse complète : docker.io/kouassihenrijoelk08/api-fil-rouge:1.1, digest sha256:9c7c10408ad715563326ffeb2dca83288f6346acb2dac1f8591afc893bbedaae). Le dépôt est public.

Q2. Pourquoi utiliser un numéro de version plutôt que latest ?

Un numéro de version identifie précisément le contenu de l'image : il permet de déployer une version déterminée, de comparer les évolutions et de revenir à une version précédente. L'étiquette latest est mobile : elle peut pointer vers un contenu différent après une nouvelle publication, ce qui rend les déploiements moins reproductibles.

Q3. Quel espace disque docker image prune a-t-il libéré ? Pourquoi nettoyer aussi les registres cloud ?

docker image prune -a -f a libéré 40,95 MB (« Total reclaimed space »). Selon docker system df, l'espace occupé par les images est passé de 352,4 MB à 149,8 MB, et les conteneurs ont été ramenés à 0. Dans le cloud, nettoyer les registres permet de supprimer les anciennes images inutilisées, de limiter le stockage consommé, de réduire les coûts éventuels et de faciliter la gestion des versions.

Conclusion générale

Ce TP nous a permis de découvrir le fonctionnement pratique de Docker dans une machine virtuelle Ubuntu Server. Nous avons téléchargé une image, créé et inspecté des conteneurs, publié un port, consulté les journaux et les processus, puis conservé des données grâce à un volume.

La comparaison avec la VM met en évidence la différence de principe : une machine virtuelle exécute un système complet avec son propre noyau, tandis qu'un conteneur isole une application et

ses dépendances en partageant le noyau Linux de la VM. Nos mesures le confirment : environ 23 fois plus rapide à démarrer et environ 24 fois moins de mémoire utilisée pour un conteneur que pour la VM.

La conteneurisation de l'API Flask a ensuite montré la construction d'une image reproductible, la réutilisation du cache, l'exécution de plusieurs instances d'une même image et la publication dans un registre, ici Docker Hub.